

COSMOS: Taming Heterogeneous Resource Demands in Serverless Functions with Complementary Swarming and Monotonic Scale-up

Yunshan Jia, Junyi Shu, *Member, IEEE*, Chiheng Lou, Fangyue Liu, Chenbo Bi, and Xin Jin, *Senior Member, IEEE*

Abstract—Serverless computing is an increasingly popular paradigm built on top of a data center network (DCN), offering scalability and fine-grained billing. However, serverless platforms must efficiently manage resources for a multitude of diverse functions and requests with different inputs. Conventional approaches often ignore the resource requirement differences between functions and individual requests, leading to resource fragmentation, poor cluster utilization, and high overhead from instance initializations.

This paper introduces COSMOS, a novel serverless computing system that improves efficiency across nodes in a cluster by exploiting the complementary resource usage patterns of different functions. Our approach is founded on two key optimizations. First, we route functions with complementary resource requirements to the same node to maximize node utilization, while simultaneously grouping instances of the same function to improve locality and reduce associated overhead. Second, we leverage the emerging capability to resize an instance's resource allocation, allowing our system to reuse existing warm instances for requests with heterogeneous resource demands and minimize instance initializations. Experiments on real-world workloads show our system improves utilization of multiple resources by 41.41%–67.01%, and increases overall throughput by up to 2.03 $\times$ compared to state-of-the-art serverless computing systems.

Index Terms—Serverless computing, Resource-aware scheduling, Heterogeneous resource utilization.

I. INTRODUCTION

Serverless computing is an emerging cloud paradigm built upon the backbone of the Data Center Network (DCN). It abstracts away server management, allowing developers to focus solely on application logic [1]–[3]. In this model, developers deploy programs as cloud functions, and the platform automatically manages the underlying infrastructure, including resource provisioning, auto-scaling and traffic routing. The simplicity and efficiency of serverless computing have led to widespread adoption in domains including web applications [4], data processing pipelines [5], [6], and machine learning workloads [7], [8]. For cloud providers, however, transforming serverless computing into a sustainable business

model necessitates efficient resource utilization across a multi-tenant server fleet.

To achieve high resource utilization, prevailing approaches typically optimize for proxy objectives, such as balancing load [9]–[11] or maximizing the reuse of initialized execution environments (often termed warm instances) [9], [12]–[16]. However, by largely ignoring the diverse resource footprints (*e.g.*, CPU-to-memory ratios) across different functions, these strategies can make sub-optimal routing decisions. If a router sends many memory-intensive functions to a single node, it risks exhausting available memory while letting CPUs remain underutilized. This fragmentation creates stranded resources, preventing the node from processing new requests and thereby lowering overall cluster utilization.

Furthermore, existing resource allocation strategies predominantly operate at the function level, where a static resource capacity is allocated to each function instance [12], [17]–[24]. While prior work recognizes that resource demands can vary across requests [23], [25]–[28], its solutions often rely on predicting an aggregate demand, selecting a single optimal static configuration [25]–[27], or applying coarse-grained bin-packing [23]. Consequently, these strategies remain fundamentally static and cannot efficiently adapt to fine-grained, input-dependent workload variations at the request level. This one-size-fits-all approach inevitably leads to inappropriate resource allocation, as it either over-provisions resources for light requests or under-provisions for heavy ones.

We confirm these inefficiencies by conducting a comprehensive analysis over a wide range of real-world serverless applications (§II-B). Our analysis shows that: i) The CPU-to-memory ratio (core/GB) varies significantly across different functions, ranging from 0.09 to 10.10, with a coefficient of variation (CV) of 62.43%. This high inter-function heterogeneity leads to significant resource stranding under existing resource allocation strategies that ignore such diversity. ii) At the request level, 54% of functions exhibit substantial variation in resource usage under different inputs, with a CV exceeding 50%. This degree of variability far exceeds what static allocation can efficiently accommodate, forcing over-provisioning to avoid performance degradation and resulting in low resource utilization.

To overcome these inefficiencies, we present COSMOS, a serverless computing system that handles the heterogeneous resource demands of functions and requests with a two-pronged solution. At the inter-node level, we introduce complementary swarming, a function request routing strategy that mitigates resource stranding by co-locating functions with opposing

Yunshan Jia, Chiheng Lou, Fangyue Liu, and Xin Jin are with the School of Computer Science, Peking University, Beijing, China. E-mail: {jiayunshan;louchiheng23;lfy-wanqiu;xinjinpku}@pku.edu.cn.

Chenbo Bi is with Yuanpei College, Peking University, Beijing, China. E-mail: 2300017783@stu.pku.edu.cn

Junyi Shu is with University of California, Los Angeles, California, USA. E-mail: kyleshu@ucla.edu.

This work was supported in part by the Scientific Research Innovation Capability Support Project for Young Faculty under grant ZYGXQNJSKYCXNLZCXM-II, and US National Science Foundation under grants CNS-2403254, CNS-2330831, and CNS-2106838. (Corresponding author: Junyi Shu.)

resource requirements while preserving locality. At the intra-node level, we design a monotonic scale-up request scheduling policy that reorders the request queue for a function to enforce monotonic resource adjustment which avoids the high overhead of instance restarts. Together, these techniques allow COSMOS to significantly improve cluster utilization.

First, to address cluster-wide fragmentation, the complementary swarming routing strategy classifies functions based on their distinct resource profiles (*e.g.*, CPU-intensive vs. memory-intensive). It employs a multidimensional packing algorithm that treats resources as distinct dimensions, intentionally co-locating functions with opposing resource needs on the same physical node. This ensures that a node's full capacity is utilized across all dimensions simultaneously, preventing one resource from becoming a bottleneck while another sits idle.

Second, to address the overhead of handling input-dependent variance, COSMOS utilizes Kubernetes' in-place resizing capability [29], which effectively facilitates unidirectional scaling; while capacity can be seamlessly increased, reducing resources still requires a costly container restart. To minimize container restarts, we introduce monotonic scale-up request scheduling, which organizes function requests into queues sorted by monotonically increasing resource demands. A warm instance processes requests sequentially from a single queue, ensuring that the container only needs to scale up—never down—during a batch. This avoids the latency of cold starts and the overhead of frequent memory reclamation, effectively amortizing the restart cost across a batch of requests.

The key contributions of this paper are as follows:

- We conduct a comprehensive characterization of real-world serverless workloads to determine and quantify the resource inefficiency caused by the heterogeneous resource demands of functions and requests.
- We design COSMOS, a serverless computing system that co-locates functions with complementary resource requirements and dynamically adjusts resources at the request level with complementary swarming for request routing and monotonic scale-up for request scheduling.
- We implement and evaluate COSMOS on real-world workloads, demonstrating 41.41%–67.01% utilization improvement across multiple resources, and up to $2.03\times$ overall throughput compared to state-of-the-art systems.

II. BACKGROUND AND MOTIVATION

A. Resource Management of Serverless Computing

Serverless computing has emerged as a promising cloud paradigm, allowing developers to focus on application logic by abstracting away infrastructure tasks such as provisioning, scaling, and maintenance. In this deployment model, users simply provide code (functions) with basic resource specifications and triggers. The platform then orchestrates the entire lifecycle—provisioning sandboxed environments, performing on-demand autoscaling, and managing pay-per-use billing. Consequently, efficient resource management is essential to ensure high performance, maximize utilization, and minimize costs for cloud platforms.

In practice, resource management for a serverless computing cluster is architected along two orthogonal axes: i) vertical

sizing, which defines the resource limits for function instances; and ii) horizontal orchestration, which governs autoscaling (instance count) and placement (node selection) of each function.

Instance resource allocation (vertical). Existing serverless computing systems from both industry [17]–[19] and academia [5], [12], [15], [30] allocate a uniform amount of resources for all instances of a given function. In AWS Lambda, a developer specifies a memory size (*e.g.*, 128 MB–10,240 MB), from which the platform derives proportional vCPU and network bandwidth shares [17]. These limits remain fixed throughout the instance lifetime. Similar static per-instance specifications are used by Azure Functions [18] and Google Cloud Run [19]. This approach presumes uniform resource demand across function requests. Consequently, existing optimizations focus on profile-guided configuration tuning [12], [15], [20], [21], [28]—by profiling across different configurations, they quantify the latency–cost trade-off and select the configuration with the minimum cost that satisfies a user-given SLO.

Instance scaling and placement (horizontal). Existing serverless computing systems perform autoscaling and placement based on a static per-function resource requirement [31]–[33]. While autoscalers adjust instance counts in response to load, schedulers employ a resource accounting model where each node tracks allocated versus available capacity. The resource counters are updated only during lifecycle events—decremented upon placement and incremented upon termination—ignoring actual usage during execution. An instance is admitted to a node only if the available capacity meets the function's static requirement (*e.g.*, memory, CPU). For instance autoscaling and placement, recent research has pursued two main optimizations: i) for autoscaling, using prediction to right-size replica counts and prewarm instances [15], [23]; and ii) for placement, improving efficiency through cache-aware [34], heterogeneity-aware [14], and data-affinity strategies [23], [35].

Taken together, the prevailing serverless paradigm relies on modeling each function with a static, uniform resource profile. However, our characterization reveals that this static paradigm conflicts with a dual-layered heterogeneity in resource demands: diverse resource profiles across functions and varying demands within each function. In the next two subsections, we quantify these heterogeneities (§II-B) and then analyze the system-level challenges that they pose to dynamic resource allocation (§II-C).

B. Analysis of Resource Requirements of Functions

Resource Demand Variation Across Functions. The resource demands of serverless functions are highly heterogeneous. We quantify this heterogeneity by analyzing the execution traces of 162 real-world functions from the open-source Huawei dataset [36], calculating the average CPU-to-memory ratio (core/GB) for each. Figure 1 presents the distribution of these ratios across all functions.¹ The distribution is highly dispersed, with ratios ranging from 0.09 to 10.10 and a high CV of

¹The statistical results shown are from [Request Table Module, Region 4] part in dataset [36]. We observed similar distributions across different parts of the dataset.

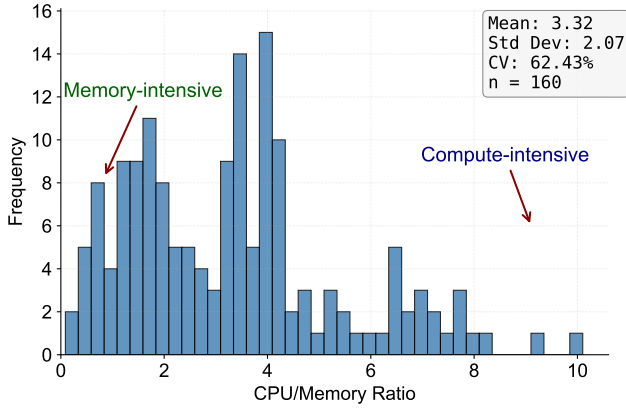


Figure 1. Heterogeneous CPU:memory ratios across functions.

62.43%, indicating that some functions are extremely compute-intensive while others are predominantly memory-intensive. This substantial variation in intrinsic resource profiles implies that a resource allocation strategy that is unaware of such differences is prone to creating resource fragmentation.

Opportunity #1: Heterogeneity-aware function placement.

The substantial dispersion in CPU-to-memory ratios across functions creates a significant opportunity to enhance cluster-wide resource efficiency through smarter placement. By leveraging the diverse resource profiles identified in our characterization, a scheduler can strategically co-locate functions with complementary demands—pairing compute-intensive workloads with memory-intensive ones. This complementary packing balances resource consumption along multiple dimensions at the node level, thereby minimizing fragmentation, maximizing aggregate utilization, and significantly increasing the effective capacity of the server fleet.

Resource Demand Variation Across Requests. Resource demand also varies significantly across different requests of the same function. We compute the coefficient of variation (CV) for the CPU usage of each function across its request trace. The distribution of these CV values is presented in Figure 2. Our analysis shows that variability is the norm rather than the exception: for 54% of functions, the CV exceeds 50%, and an additional 26% have a CV between 15% and 50%. The exact cause of this variation can be multifaceted, including hardware heterogeneity, communication patterns, or—the primary and widely acknowledged source—variability in request inputs (*e.g.*, parameters or data size). Regardless of the specific cause, this widespread and often substantial variability directly contradicts the assumption of uniform demand underlying static allocation. Consequently, any static resource allocation must provision for the peak demand of such variable functions, inevitably leading to low average utilization.

Opportunity #2: Dynamic in-place runtime resource adjustment. A key opportunity to address the intra-function variability lies in recent container runtime enhancements that support dynamic, in-place resource adjustment. Specifically, platforms like Kubernetes now allow the resource allocations (CPU and memory) of a running pod to be updated without restarting it, via mechanisms such as the Pod resize subresource and live

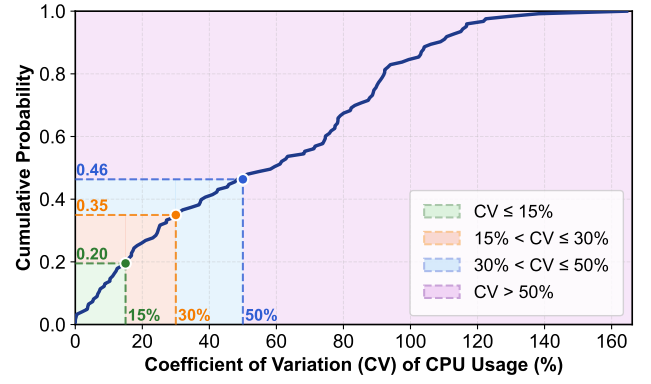


Figure 2. Resource demand variations of serverless function.

cgroup updates [29], [37]. This capability enables a container’s assigned resources to be aligned in real time with the actual demand of the incoming request. It thus provides a foundational mechanism to move beyond static per-function allocation, toward a fine-grained model where resources can be expanded or contracted per request. It absorbs demand bursts, reduces stranded resources from over-provisioning, and lowers the risk of SLO violation—while preserving performance benefits of a warm instance.

C. Challenges in Dynamic Resource Allocation for Functions

Challenge #1: Placement under dynamic in-place resizing.

Complementary placement addresses resource fragmentation caused by ignoring heterogeneous resource profiles (*e.g.*, CPU-to-memory ratios). However, dynamic in-place resizing disrupts the straightforward fixed-size bin-packing model: a sandbox’s footprint may expand while active, making node feasibility assessments at start/stop times inadequate. A simplistic approach is to evict co-located instances to make space when upscaling faces insufficient capacity. However, this naive strategy risks evicting other co-located function instances with increased resource demand, potentially resulting in eviction amplification: a single significant resize event can force the eviction of multiple smaller co-located instances, thereby hurting cache locality and increasing tail latency. Additionally, fragmentation can occur when resource demand decreases. These problems may lead to low resource utilization and application throughput.

Therefore, an intelligent placement strategy must anticipate and mitigate the disruptive effects of resizing. The goal is twofold: i) to balance CPU and memory usage per node from the outset, preventing resource fragmentation and minimizing the need for disruptive migrations; and ii) to migrate instances in a manner that minimizes overall disruption when necessary. By promoting managed, minimal churn, such a strategy safeguards warm states (maximizing instance reuse) and maintains high resource utilization even during dynamic resizing.

Challenge #2: High overhead of downsizing. Dynamic in-place scaling is asymmetric: upsizing is swift, but downsizing incurs high costs. Increasing the memory allocation takes effect immediately, whereas reducing it triggers a restart which involves cgroup reclamation, page-cache eviction, and compaction, potentially stalling both runtime and kernel. Since resizing is on the critical path of request handling, this

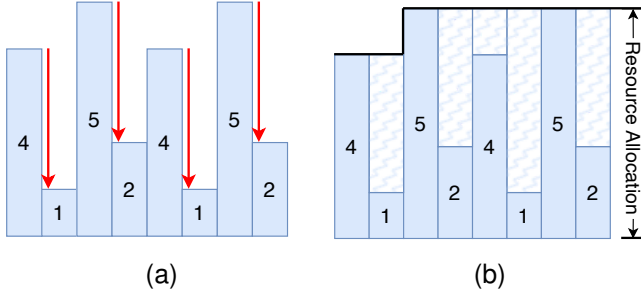


Figure 3. Inefficient scaling strategies under variable workloads. (a) Eager bidirectional resizing with prohibitive reclamation overheads. (b) Increment-only resizing with wasted headroom.

expensive downsizing process can directly degrade both overall throughput and end-to-end latency.

This asymmetry creates a fundamental dilemma for naive instance resizing strategies. If an instance undergoes eager bidirectional resizing for every request, the high downsizing costs identified above incur prohibitive memory reclamation overheads and disrupt page caches and JIT states. Conversely, a increment-only strategy that avoids downsizing inevitably leads to severe over-provisioning, as instances accumulate resources to match their historical peak demand. In either case, the theoretical advantages of dynamic scaling are negated. These inefficiencies are illustrated in Figure 3, which contrasts the reclamation overhead of bidirectional resizing with the wasted headroom of increment-only resizing.

Practical dynamic resource allocation must achieve the following goal: it should adapt resource allocation to varying request-level demands without incurring significant overheads of frequent downsizing. This necessitates a mechanism that can batch or reorder requests to create resource usage patterns that are favorable to increment-only adjustments within short time windows, thereby amortizing reclamation costs and preserving warm instance state.

III. COSMOS OVERVIEW

Figure 4 illustrates the overall architecture of COSMOS, which operates at two levels: inter-node request routing and intra-node request scheduling. At the cluster level, incoming user requests are directed to a request router, which monitors traffic patterns and forwards these statistics, along with function metadata and performance metrics, to the routing plan generator that periodically produces routing plans used to assign requests to nodes. Once assigned to a worker node, requests are entered into queues managed by the worker scheduler. This component maintains separate queues and priorities for different functions, while orchestrating instances within the node to handle request execution and in-place resource scaling. To enable this, COSMOS introduces two key techniques: complementary swarming routing and monotonic scale-up scheduling.

Complementary Swarming Request Routing. COSMOS controls instance placement by establishing a mapping dictating how requests are distributed across the cluster. Using a heuristic algorithm, COSMOS ensures that the collective resource demand of the function mix on each worker node aligns

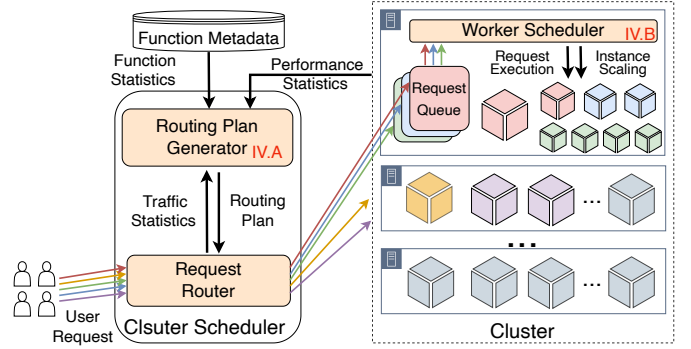


Figure 4. COSMOS overview.

with the node's capacity, thereby balancing CPU and memory usage per node and preventing resource fragmentation. For high-demand functions requiring deployment across multiple nodes, COSMOS employs request-level load splitting that groups requests of similar resource demand onto the same node where possible. This size-aware segregation mitigates the eviction amplification risk under dynamic in-place resizing by preventing uncontrolled, concurrent upsizing across nodes. By controlling the optimization step size and applying partial updates, COSMOS keeps the average algorithm duration under 110 milliseconds.

Monotonic Scale-up Request Scheduling. COSMOS designs its worker-level scheduling algorithm around the restart-free in-place pod scaling feature of Kubernetes. The scheduler maintains dual request queues, both sorted in ascending order of resource demand. This ordering ensures that, within a time window, the resource requirements of requests processed by an instance increase monotonically. Consequently, an instance can be scaled up incrementally to meet the growing demands of successive requests without over-provisioning, thereby increasing resource utilization without incurring restarts. To prevent starvation, COSMOS determines queue priorities by holistically considering request queuing time, cold-start overhead, and resource requirements. By strategically reordering requests under priority constraints, COSMOS effectively boosts resource utilization and reduces the frequency of cold starts.

IV. COSMOS DESIGN

COSMOS employs a two-level hierarchical orchestration approach for serverless function and request: complementary swarming request routing at the inter-node level and monotonic scale-up request scheduling at the intra-node level.

A. Complementary Swarming Request Routing

The complementary swarming routing algorithm addresses the inter-node resource fragmentation posed by heterogeneous function resource demands, as quantified in §II-B. It pursues a dual objective: i) maximizing cluster-wide resource utilization by co-locating functions with complementary CPU and memory profiles on the same node, thereby mitigating resource stranding; and ii) enhancing instance reuse by deliberately narrowing the set of distinct functions served per node. To achieve this, the algorithm proceeds iteratively. In each round, it

assigns the workload of a function to one or more worker nodes based on the function's resource profile and, when necessary, rolls back partial assignments of other already-placed functions to improve complementary packing. For functions requiring distribution across multiple nodes, the algorithm performs fine-grained load splitting according to the function resource demand distribution. The algorithm outcome is a stable request-to-node mapping that balances complementary packing with demand-aware load distribution.

1) *Modeling Node Capacity and Function Demand*: A precise formulation of the routing problem requires clear representations of both node resource capacity and function resource demand.

Node Resource Capacity. We model a worker node N_j 's resource capacity as a two-dimensional vector $\mathbf{C}_j = (C_j^{\text{cpu}}, C_j^{\text{mem}})$, representing its CPU cores and memory (in GB). Similarly, we denote by $\mathbf{C}_j^{\text{idle}} = (C_j^{\text{idle,cpu}}, C_j^{\text{idle,mem}})$ the vector of idle resources on N_j at any given moment. For simplicity, we focus on CPU and memory as the primary bottleneck resources. Our algorithm can be extended to incorporate other resources (e.g., network bandwidth) by expanding the resource vector. We define the **overall resource utilization** of a node N_j as the sum of its CPU and memory utilization:

$$U(N_j) = \frac{C_j^{\text{cpu}} - C_j^{\text{idle,cpu}}}{C_j^{\text{cpu}}} + \frac{C_j^{\text{mem}} - C_j^{\text{idle,mem}}}{C_j^{\text{mem}}}$$

Here, we simply sum CPU and memory utilization because function demands will be standardized to eliminate weight differences between resources. An optimization step is considered beneficial if it increases $U(N_j)$.

Function Resource Demand. Modeling the resource demand of a function is more nuanced. We characterize it along three orthogonal dimensions:

- 1) **Demand Ratio** (r_i): The intrinsic CPU-to-memory ratio required by a single execution of function F_i . The value of r_i quantitatively indicates a function's resource affinity: a relatively small r_i signifies memory-intensive behavior, whereas a relatively large r_i corresponds to compute-intensive behavior. To reduce problem complexity, we assume this ratio remains invariant across requests of the same function.
- 2) **Demand Volume** ($\mathbf{V}_i$): The aggregate resource demand vector for function F_i within the scheduling window, denoted as $\mathbf{V}_i = (V_i^{\text{cpu}}, V_i^{\text{mem}})$. Here, V_i^{cpu} and V_i^{mem} represent the predicted total CPU (e.g., in core-seconds) and memory (e.g., in GB-seconds) required to process all expected requests of F_i in that window, respectively. While the accurate prediction of V_i is orthogonal to our placement algorithm (and its accuracy naturally affects overall performance), our design accepts such a predicted vector as input.
- 3) **Demand Distribution** (D_i): The statistical distribution of per-request resource requirements, derived from historical traces. D_i is crucial for fine-grained load splitting when a function must be spread across multiple nodes.

Together, the tuple $\langle r_i, \mathbf{V}_i, D_i \rangle$ forms a comprehensive demand profile for each function F_i , enabling our algorithm to rea-

Algorithm 1 Complementary swarming routing.

Data:

- Pending assignment vector $\mathbf{R} \in [0, 1]^n$.
- Assignment matrix $\mathbf{L} \in [0, 1]^{m \times n}$.

Require:

- Node set $\mathcal{N} = \{N_1, \dots, N_m\}$.
- Function set $\mathcal{F} = \{F_1, \dots, F_n\}$.

Ensure: Optimized routing map $\mathcal{P} : \mathcal{F} \times [0, 1] \rightarrow \mathcal{N}$.

```

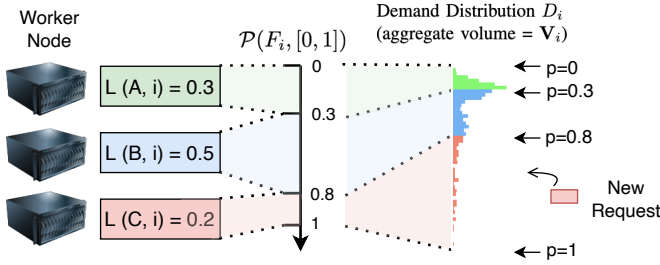
1: // Step 1: Initialize state & normalize function demands
2:  $\mathbf{L} \leftarrow \mathbf{0}_{m \times n}$ ,  $\mathbf{R} \leftarrow \mathbf{1}_n$ 
3: for  $F_i \in \mathcal{F}$  do
4:    $\mathbf{V}_i \leftarrow \mathbf{V}_i \times \frac{\sum_{j=1}^m C_j}{\sum_{i=1}^n V_i}$   $\triangleright$  Scale demand proportionally
5:
6: // Step 2: Main optimization loop
7: while  $\exists F_i$  s.t.  $R[i] > 0$  do  $\triangleright$  Mapping incompleteness
8:    $F_a \leftarrow \text{RandomChoice}(\{F_i \mid R[i] > 0\})$ 
9:   for  $N_j \in \mathcal{N}$  do
10:    // Assignment via free capacity
11:     $\alpha_j^{\text{free}} \leftarrow \min(\frac{C_j^{\text{idle,cpu}}}{V_i^{\text{cpu}}}, \frac{C_j^{\text{idle,mem}}}{V_i^{\text{mem}}})$ 
12:    // Assignment via beneficial swaps
13:     $(\alpha_j^{\text{swap}}, \mathcal{E}_j) \leftarrow \text{SwapCapacity}(F_a, N_j, \mathbf{L}, \mathbf{C}_j^{\text{idle}})$   $\triangleright$ 
     $\mathcal{E}_j$ : list of  $(F_k, \beta_k)$  evicted
14:     $\alpha_j^{\text{max}} \leftarrow \min(\alpha_j^{\text{free}} + \alpha_j^{\text{swap}}, R[a])$ 
15:     $\Delta U_j \leftarrow \text{UtilGain}(F_a, \alpha_j^{\text{max}}, \mathcal{E}_j, \mathbf{C}_j)$ 
16:    // Decision & Assignment (ignoring edge cases)
17:    for  $N_j \in \mathcal{N}$ , descending order of  $\Delta U$  do
18:      if  $R[a] \leq 0$  then break
19:       $L[j, a] \leftarrow L[j, a] + \alpha_j^{\text{max}}$ ,  $R[a] \leftarrow R[a] - \alpha_j^{\text{max}}$ 
20:      for  $(F_k, \beta_k)$  in  $\mathcal{E}_j$  do
21:         $L[j, k] \leftarrow L[j, k] - \beta_k$ ,  $R[k] \leftarrow R[k] + \beta_k$ 
22:
23: // Step 3: Generate  $\mathcal{P}$  from  $L$ 
24: for  $F_i \in \mathcal{F}$  do
25:    $p_{\text{start}} \leftarrow 0.0$   $\triangleright$  To store workload percentile range
26:   for  $N_j \in \mathcal{N}$ , where  $L[j, i] > 0$ , in random order do
27:     // We use half-open intervals  $[p_s, p_e)$ , except the
    last interval which is closed at 1.
28:      $\mathcal{P}(F_i, [p_{\text{start}}, p_{\text{start}} + L[j, i])) \leftarrow N_j$ 
29:      $p_{\text{start}} \leftarrow p_{\text{start}} + L[j, i]$ 
30: return  $\mathcal{P}$ 

```

son about complementary routing and proportional workload distribution simultaneously.

2) *Complementary Swarming Routing Algorithm*: Algorithm 1 presents the complementary swarming routing workflow. It takes as input the set of nodes $\mathcal{N}$ with their capacity vectors $\mathbf{C}_j$, and the set of functions $\mathcal{F}$ with their demand profiles $\langle r_i, \mathbf{V}_i, D_i \rangle$. The algorithm outputs a routing map $\mathcal{P} : \mathcal{F} \times [0, 1] \rightarrow \mathcal{N}$. This mapping determines the request routing rule: when the workload of function F_i is split across multiple nodes, $\mathcal{P}$ assigns each request from F_i to a specific node based on the request's percentile in the function demand distribution D_i . This percentile-based splitting is illustrated in Figure 5.

Step 1: Initialization & Demand Normalization. The algorithm maintains two key data structures initialized in this

Figure 5. Generate $\mathcal{P}$ from L .

step: i) a pending assignment vector $\mathbf{R}$, where $R[i]$ denotes the fraction of F_i demand yet to be assigned, initialized to 1 for all functions; and ii) an assignment matrix $\mathbf{L}$, where $L[j, i]$ records the fraction of F_i demand currently assigned to node N_j , initialized to all zeros (line 2). Furthermore, we scale the demand vector $\mathbf{V}_i$ of each function proportionally so that the aggregate demand matches the total cluster capacity ($\sum_i \mathbf{V}_i = \sum_j \mathbf{C}_j$). This normalization ensures the subsequent routing operates under a balanced supply-demand premise (Line 3–4).

Step 2: Iterative Optimization Loop. The core of the algorithm is an iterative loop that runs until all function demands are placed ($R[i] = 0, \forall i$, Line 7). In each iteration, it randomly selects a function F_a with pending demand ($R[a] > 0$, Line 8). For each node N_j , the algorithm evaluates two ways to accommodate F_a : i) using the idle node resources, which yields a capacity α_j^{free} computed as the minimum ratio of idle CPU/memory to F_a demand (Line 11); and ii) through beneficial swaps, where the subroutine SwapCapacity attempts to evict already-placed functions in order to host as much of F_a demand as possible, provided that doing so improves the node utilization U_j (Line 13, detailed in Algorithm 2). SwapCapacity returns both the maximum F_a demand fraction α_j^{swap} that can be hosted by function swapping, and a list $\mathcal{E}_j$ of evicted functions, each with the evicted load fraction β . The total accommodation capacity for F_a on N_j is then $\alpha_j^{\text{max}} = \min(\alpha_j^{\text{free}} + \alpha_j^{\text{swap}}, R[a])$ (Line 14).

After evaluating all nodes, the algorithm sorts them in descending order of the utilization gain ΔU_j (Line 17) and assigns F_a pending demand greedily. For each node in this order, it assigns up to α_j^{max} of F_a demand, updates $\mathbf{L}$ and $\mathbf{R}$ (Line 19), and simultaneously processes the evictions in $\mathcal{E}_j$: for each function F_k in $\mathcal{E}_j$, the algorithm reduces the evicted fraction β_k in $\mathbf{L}[j, k]$ and adds β_k back to F_k pending demand $\mathbf{R}[k]$ (Line 21). This process continues until F_a demand is fully assigned ($R[a] = 0$). Because the total function demand was scaled to match the total cluster capacity in Step 1, it is guaranteed that F_a demand can eventually be fully accommodated, and the loop will not terminate with $R[a] > 0$. To ensure the loop convergence speed, the algorithm skips nodes whose utilization gain ΔU_j is below a predefined empirical threshold (e.g., 1%).

Step 3: Fine-Grained Routing Plan Generation. After the assignment matrix $\mathbf{L}$ is determined, the algorithm converts the proportional assignments into a precise routing map $\mathcal{P}$ (Line 24–29). This process is illustrated in Figure 5. For each function F_i , the algorithm sequentially allocates contiguous

Algorithm 2 SwapCapacity: Compute beneficial swap capacity

Require:

- Node N_j with idle resource vector $\mathbf{C}_j^{\text{idle}}$
- Target function F_a
- Allocation matrix $\mathbf{L}$

Ensure: Swap capacity α^{swap} , eviction list $\mathcal{E} = [(F_k, \beta_k)]$

```

1:  $\alpha^{\text{swap}} \leftarrow 0, \mathcal{E} \leftarrow [], \{F_k\} \leftarrow \{F_k | L[j, k] > 0\}$ 
2: if  $C_j^{\text{idle,cpu}} > 0$  then  $\triangleright$  Determine dominant idle resource
3:   // Replace more memory-intensive first
4:   sort  $\{F_k\}$  by  $r_k$  ascending; sign  $\leftarrow +1$ 
5: else  $\triangleright C_j^{\text{idle,mem}} > 0$ 
6:   // Replace more compute-intensive first
7:   sort  $\{F_k\}$  by  $r_k$  descending; sign  $\leftarrow -1$ 
8: for each  $F_k$  in the sorted order do
9:   if sign  $\cdot (r_a - r_k) \leq 0$  then
10:    continue  $\triangleright$  Swap would not improve  $U(N_j)$ 
11:   Compute max  $\beta_k$  s.t. evicting  $\beta_k$  of  $F_k$  respects  $\mathbf{C}_j^{\text{idle}}$ 
12:    $\beta_k \leftarrow \min(\beta_k, L[j, k])$ 
13:    $\gamma_k \leftarrow \beta_k \cdot (\mathbf{V}_k \oslash \mathbf{V}_a) \triangleright \gamma_k = \beta_k \cdot \min\left(\frac{V_k^{\text{cpu}}}{V_a^{\text{cpu}}}, \frac{V_k^{\text{mem}}}{V_a^{\text{mem}}}\right)$ 
14:   if  $\alpha^{\text{swap}} + \gamma_k > R[a]$  then
15:      $\gamma_k \leftarrow R[a] - \alpha^{\text{swap}}$ ; adjust  $\beta_k$  proportionally
16:    $\alpha^{\text{swap}} \leftarrow \alpha^{\text{swap}} + \gamma_k$ 
17:    $\mathcal{E} \leftarrow \mathcal{E} \cup \{(F_k, \beta_k)\}$ 
18:   if  $\alpha^{\text{swap}} \geq R[a]$  then break
19: return ( $\alpha^{\text{swap}}, \mathcal{E}$ )

```

percentile intervals in $[0, 1]$ to each node N_j that hosts F_i (left side of the figure). The length of the interval assigned to N_j is exactly $L[j, i]$. This completes the construction of $\mathcal{P}$, which is the algorithm's final output. In practice, to avoid resource fragmentation that could arise if certain nodes only receive large requests, the intervals are assigned to nodes in a randomized order rather than a fixed sequence; the figure omits this step for simplicity.

Request Routing. At runtime, the interval $[0, 1]$ is concretely mapped to the resource-demand percentiles of F_i 's requests (right side of Figure 5). Specifically, using F_i 's historical demand distribution D_i , we pre-compute the resource-demand boundaries that correspond to each percentile sub-interval. When a new request arrives, its actual resource demand (derived from input parameters and data size) is located on D_i to obtain its percentile p . The request is then routed to the node N_j for which $\mathcal{P}(F_i, p) = N_j$. For example, the newly arrived request marked in the figure, whose demand percentile falls inside $[0.8, 1]$, would be routed to Node C. Our experiments obtain accurate demand estimates from the benchmark's known input characteristics; the algorithm can be coupled with more sophisticated prediction methods for further performance gains.

Assignment via beneficial swaps. The SwapCapacity subroutine (Algorithm 2) determines, for a given node N_j and a target function F_a , the maximum additional fraction α_j^{swap} of F_a demand that can be hosted by beneficially evicting already-placed functions. A swap is considered beneficial only if it increases the overall node utilization $U(N_j)$. This requirement dictates that swaps must consume the node's dominant idle

resource: if CPU is idle, F_a must replace more memory-intensive functions (with smaller demand ratio r_k); if memory is idle, it must replace more compute-intensive ones (with larger r_k). The subroutine takes as input the node idle resource vector $\mathbf{C}_j^{\text{idle}}$, the target function F_a , and the current assignment matrix $\mathbf{L}$; it returns the admissible swap capacity α^{swap} together with a list $\mathcal{E}$ of evicted functions, each annotated with the evicted load fraction β_k .

The subroutine proceeds in four logical steps. First, it identifies the dominant idle resource—CPU or memory—since after using the node’s free capacity (line 11 of Algorithm 1), only one resource dimension may remain idle (lines 2–7). This determines the order in which existing functions are examined: if CPU is idle, functions are sorted by their resource-demand ratio r_k in ascending order (*i.e.*, more memory-intensive functions first); if memory is idle, they are sorted in descending order (more compute-intensive ones first). This ordering ensures that the most beneficial candidates are tried first, because replacing a function with a very complementary resource profile allows F_a to consume more of the idle resource.

Second, for each candidate function F_k in this sorted list, the algorithm performs a benefit test (line 9). The condition $\text{sign} \cdot (r_a - r_k) > 0$ guarantees that swapping a fraction of F_k for F_a will actually increase $U(N_j)$. Intuitively, if CPU is idle ($\text{sign} = +1$), we must have $r_a > r_k$, meaning F_a is more compute-intensive than F_k and will better utilize the spare CPU; the converse holds when memory is idle. Candidates that fail this test are skipped.

Third, for a candidate that passes the test, the algorithm computes how much of F_k can be evicted (β_k) without exceeding the node’s idle resources, and limits β_k by the amount currently assigned to N_j ($L[j, k]$) (lines 11–12). It then translates this evicted capacity into an equivalent amount of F_a that can be hosted using the freed resources. The translation uses the formula $\gamma_k = \beta_k \cdot \min\left(\frac{V_k^{\text{cpu}}}{V_a^{\text{cpu}}}, \frac{V_k^{\text{mem}}}{V_a^{\text{mem}}}\right)$ (line 13), which ensures that neither CPU nor memory exceeds the vacated capacity.

Finally, the algorithm accumulates the admissible fractions γ_k until either the pending demand $R[a]$ of F_a is exhausted or no further beneficial swaps can be found (lines 14–17). The accumulated total α^{swap} and the corresponding eviction list $\mathcal{E}$ are returned to the main loop in Algorithm 1.

Low Overhead via Partial Updates. To keep the scheduling overhead low, the routing algorithm employs an incremental (partial) update mechanism. Rather than recomputing the full routing plan from scratch for every scheduling window, the system periodically (every minute in our implementation) monitors the latest demand profiles $\langle r_i, \mathbf{V}_i, D_i \rangle$ of all functions. For a function whose aggregate demand $\mathbf{V}_i$ has not increased and demand distribution D_i remains statistically stable, the algorithm preserves its current placement. Only functions with significantly changed demand (increased $\mathbf{V}_i$ or altered D_i) are fed into the complementary swarming algorithm to update their assignments. This partial update drastically reduces the number of functions that need to be re-routed in each iteration, which in turn minimizes instance migration, thereby reducing scheduling churn and preserving warm instance states. Furthermore, to prevent the accumulation of sub-optimality that may arise

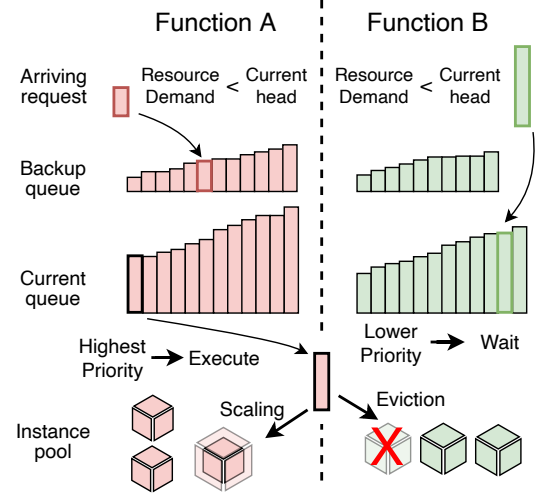


Figure 6. Monotonic scale-up scheduling workflow.

from multiple incremental updates, the system performs a global re-placement every 30 minutes, which recomputes the assignment for all functions from a clean state, ensuring long-term placement quality.

B. Monotonic Scale-up Request Scheduling

The monotonic scale-up scheduler runs on each worker node. Its primary objective is to handle the intra-function demand variability identified in §II-B by leveraging in-place vertical scaling, thereby avoiding costly cold starts while maintaining high resource efficiency.

1) *Scheduler Architecture:* As illustrated in Figure 6, the core scheduling structure is organized per function. For each function F_i , the scheduler maintains two monotonic request queues: a current queue and a backup queue. Requests in both queues are ordered by their predicted resource demand in increasing order. This monotonic ordering is the key to enabling efficient in-place scaling: when an instance processes requests sequentially from a single queue, its resource allocation can be increased incrementally without restarts.

Request-insertion rule. When a new request of F_i arrives, its predicted resource demand is compared with the demand of the head (smallest) request in the current queue. If the new demand is smaller, the request is placed into the backup queue; otherwise, it is inserted into the current queue (preserving the monotonic order). The purpose of this rule is not merely to keep each queue sorted, but to guarantee that an instance, while processing requests from a single current queue, encounters a resource demand sequence that is monotonically non-decreasing. This property is essential: if an instance were forced to run a larger request before a smaller one, it would either have to start at the higher capacity (wasting resources) or be restarted after a down-scale (incurring cold-start overhead). The backup queue temporarily holds smaller requests until the current queue is fully processed, after which the two queues swap roles.

Instance execution and priority-based selection. A function may have from zero to multiple concurrent warm instances, each dedicated to fetching requests only from that function’s

current queue. The scheduler assigns a dynamic priority to each function (detailed in §IV-B2). At every scheduling point—triggered by a new request arrival or a request completion—the scheduler selects the function with the highest priority and attempts to process the head request of its current queue.

Resource-assignment policy. To execute the selected request, the scheduler tries the following options in order, aiming to minimize overhead:

- i) **Reuse** an existing instance of the same function whose current resource allocation already meets or slightly exceeds the request's demand, thereby avoiding any scaling operation.
- ii) **In-place scaling** of a warm instance via restart-free vertical scaling (Kubernetes Pod resize API) to accommodate the request, thus preserving warm state.
- iii) **Launch a new instance** (cold start) if no suitable warm instance is available.

If the node lacks sufficient idle resources for options ii) or iii), the scheduler evaluates eviction. It selects candidate instances according to a per-sandbox eviction priority computed as:

$$\text{Prio}_{\text{instance}} = \frac{\text{Prio}_{\text{function}}}{N_{\text{instances}}},$$

where $N_{\text{instances}}$ is the number of active instances belonging to the same function. Among instances of the same function, those with larger resource footprints are evicted first. This scheme encourages load spreading (by penalizing functions that already hold many instances) and promotes balanced resource sharing. Eviction is performed only if it can actually release enough resources to execute the pending request; otherwise, the scheduler leaves the system unchanged and waits for the next scheduling event.

2) *Priority Design*: The scheduler assigns a dynamic priority to each function F_i to balance three objectives: minimizing cold-start overhead, preventing request starvation, and maximizing the utility of in-place scaling. The priority is computed as a weighted sum of three factors derived from the state of F_i 's queues:

$$\text{Priority}(F_i) = \omega_1 \cdot \text{factor}_1 + \omega_2 \cdot \text{factor}_2 + \omega_3 \cdot \text{factor}_3,$$

where the weights $\omega_1, \omega_2 > 0$ and $\omega_3 < 0$ are tunable parameters (set to 0.3, 1.1 and 0.8 respectively in our implementation). The three factors are defined as follows:

- **Cold-start cost factor** (factor_1). Let t_{last} be the request with the largest predicted resource demand in F_i 's current queue, and let $C_{\text{cold}}(F_i)$ denote the estimated cold-start cost of F_i (i.e., the time overhead of launching a new instance). Then

$$\text{factor}_1 = \max\left(\frac{t_{\text{last}}^{\text{mem}}}{C_j^{\text{mem}}}, \frac{t_{\text{last}}^{\text{cpu}}}{C_j^{\text{cpu}}}\right) \cdot C_{\text{cold}}(F_i),$$

where C_j^{mem} and C_j^{cpu} are the memory and CPU capacities of the worker node N_j on which the scheduler is running. This factor increases when the largest pending request occupies a large fraction of the node's resources and the function itself is expensive to cold-start. It encourages the

scheduler to keep executing F_i 's requests consecutively, thereby raising the chance of reusing a warm instance.

- **Starvation-avoidance factor** (factor_2). Let W_{max} be the longest waiting time among all pending requests of F_i , and let $\bar{T}(F_i)$ be the average execution duration of F_i requests. The starvation factor is zero if no request has been waiting excessively; otherwise it grows with the cumulative waiting time and the total work left in the queue:

$$\text{factor}_2 = \begin{cases} 0, & \text{if } W_{\text{max}} < 3 \cdot \bar{T}(F_i), \\ \bar{T}(F_i) \cdot |Q_i| + W_{\text{max}}, & \text{otherwise.} \end{cases}$$

Here $|Q_i|$ is the total number of requests in F_i 's current queue. This mechanism ensures that a function whose requests have been waiting too long receives a priority boost, preventing indefinite starvation.

- **Scaling-efficiency factor** (factor_3). Let t_{first} and t_{last} be the requests with the smallest and largest predicted resource demands in F_i 's current queue, respectively. Under the assumption that all requests of the same function share the same CPU-to-memory ratio (see §IV-A1), the ratio can be computed using either dimension; we use memory for concreteness:

$$\text{factor}_3 = \frac{t_{\text{first}}^{\text{mem}}}{t_{\text{last}}^{\text{mem}}}.$$

A large ratio (close to 1) indicates that the queue's resource-demand range is narrow; consequently, a current instance (if any) requires little or no additional scaling to process all requests in the current queue, resulting in low execution cost. A small ratio (i.e., a wide demand spread) implies that the current instance would need substantial or multiple scaling steps to process the entire queue, making it less beneficial to continue with the current instance.

Finally, to further promote instance reuse, the priority of any function that already possesses one or more warm instances is **doubled**. The combined priority is recomputed at every scheduling event, enabling the scheduler to adapt dynamically to changing workload patterns.

V. IMPLEMENTATION

We implement a prototype of COSMOS with ~3100 LOC in Python, using Kubernetes [38] for instance management, Minio [39] for function data management, and SciPy [40] for workload prediction.

Instance manager. We use containers as the serverless function instance based on the Kubernetes container orchestration. Each function instance is exposed via a unique Pod IP address, which the scheduler maintains in a pod-IP mapping table. Container resource limits and instance placement are enforced through Kubernetes Pod specifications and node-affinity rules, respectively. In the prototype, the scheduler must manage a large number of instances and frequently interact with each one to perform lifecycle operations—including startup, access, scaling, termination, and health monitoring. To reduce control-plane overhead, our implementation leverages the Python Kubernetes client library in a multi-process and multi-coroutine

Table I
BENCHMARK CHARACTERISTICS: RESOURCE PROFILES AND EXECUTION TIMES.

Workload	Type	CPU (cores)	Mem (GB)	Duration (s)	CPU/Mem (core/GB)	Description
graph-bfs	Scientific	1.5	1.0	3.0	1.50	Breadth-first search traversal implemented using igraph.
graph-mst	Scientific	1.5	1.0	4.2	1.50	Computing minimum spanning tree using igraph.
graph-pagerank	Scientific	4.0	4.0	21.5	1.00	PageRank algorithm execution with igraph.
dna-visualisation	Scientific	1.0	2.0	7.6	0.50	Generating visualization data from DNA sequences.
dynamic-html	Webapps	1.0	1.0	0.15	1.00	Rendering dynamic HTML content from a template.
uploader	Webapps	2.0	2.0	2.5	1.00	Uploading files from a given URL to cloud storage.
compression	Utilities	1.0	2.0	1.5	0.50	Compressing multiple files into a ZIP archive for download.
image-recognition	ML	2.5	4.0	1.6	0.63	Performing image classification using ResNet and PyTorch.
thumbnailer	Multimedia	0.5	2.0	5.0	0.25	Creating thumbnail images from input pictures.
video-processing	Multimedia	1.5	4.0	11.0	0.38	Adding watermarks and generating GIFs from video files.

architecture. Since some benchmarks require reading from and writing to external data during execution, we additionally deploy a MinIO service within the cluster to provide a unified data management layer for the function system.

Scheduler. The scheduler in the prototype consists of four modules. i) The routing plan generator runs as a periodic cluster-level service. It executes the complementary swarming routing algorithm to compute the optimized request routing plan, with an execution period of one minute, matching the instance scaling interval commonly used in industrial and open-source practice [31], [41]. ii) The request router operates as a separate cluster-level process. It assigns each incoming request to a worker node according to the latest routing plan. iii) The worker scheduler runs locally on each worker node. It maintains per-function request queues, applies the monotonic scale-up scheduling policy to determine execution order, and triggers in-place pod scaling when required. iv) The system monitor collects real-time resource usage and per-function metrics (*e.g.*, cold-start time, execution duration, resource demand distribution). For load prediction, it currently employs a basic FFT-based method implemented with the Python SciPy library [40].

VI. EVALUATION

In this section, we first evaluate the overall performance of COSMOS against state-of-the-art systems—Palette [42] and FaaSCache [12]—in §VI-B. Next, we conduct an in-depth analysis of the effectiveness of the key techniques employed in COSMOS in §VI-C. Finally, we investigate the scalability of COSMOS in large-scale cluster environments in §VI-D.

A. Evaluation Setup

Benchmarks. We selected ten benchmark applications from a recent open-source serverless benchmarking suite [43], [44] to evaluate the performance of COSMOS across diverse workloads. Table I summarizes the basic distribution and resource profiles of these benchmarks. The selected benchmarks span multiple categories—including Web Apps, Multimedia, Utilities, Machine Learning, and Scientific Computing—and exhibit substantial variation in resource requirements. As shown in the table, their memory consumption ranges from 1–4 GB, CPU usage from 0.5–4 cores, and execution time from 0.15–21.5 seconds. Crucially, the CPU-to-memory ratio (shown in the table) varies widely from 0.25 to 1.50 cores/GB, highlighting the spectrum from memory-intensive (*e.g.*, thumbnailer) to CPU-intensive (*e.g.*, graph-bfs) workloads.

Workload generation. We generate user traffic based on the Azure Functions Dataset [15]. For each benchmark, we randomly sample a function invocation trace from the top 20% most frequently invoked functions, which account for the majority of resource consumption in production environments [15], representing the critical bottleneck for cluster performance optimization. Each trace’s invocation rate is proportionally scaled so that the aggregated resource demand approaches the total cluster capacity, and the invocation frequencies across the ten benchmarks are adjusted to be nearly identical, with relative differences under 2%. Given that the original trace data is recorded at minute-level granularity, intra-minute request arrivals are modeled as a Poisson process, a standard approach for independent random events in systems and performance analysis [45].

To accurately model real-world serverless functions, we first analyze the resource usage distribution in the Azure Functions dataset [15]. Using both the Shapiro-Wilk [46] and Kolmogorov-Smirnov [47] tests for normality, we find that over 95% of functions exhibit memory usage patterns that do not significantly deviate from a normal distribution (*p*-value > 0.05). Based on this empirical observation, we model the per-function resource demand as normally distributed across requests. For our evaluation, we randomly select ten functions from the dataset and perform Gaussian fitting to estimate the variance of their memory usage. We then adjust the input data size and parameters of each benchmark so that the actual resource requirements reflect the sampled variance. Since the dataset primarily captures memory demand, CPU allocation is assumed to scale proportionally with the fixed CPU-to-memory ratio for each benchmark.

Cluster configuration. Experiments are conducted on Cloud-Lab [48] using a cluster of nine c6620 instances (56 cores total, 2× Intel Xeon Gold 5512U @ 2.1GHz, 128GB memory). Eight nodes serve as worker servers and one as the scheduler and request generator. To emulate the heterogeneous resource availability typical of production clusters, each worker node is allocated only a subset of its total resources. Unless otherwise specified, the available resource proportions across nodes are sampled from a normal distribution with a standard deviation of 0.85.

Baselines. We evaluate COSMOS against two state-of-the-art resource management approaches for serverless platforms, both of which optimize cluster efficiency from complementary perspectives. For Palette, each benchmark is assigned a unique

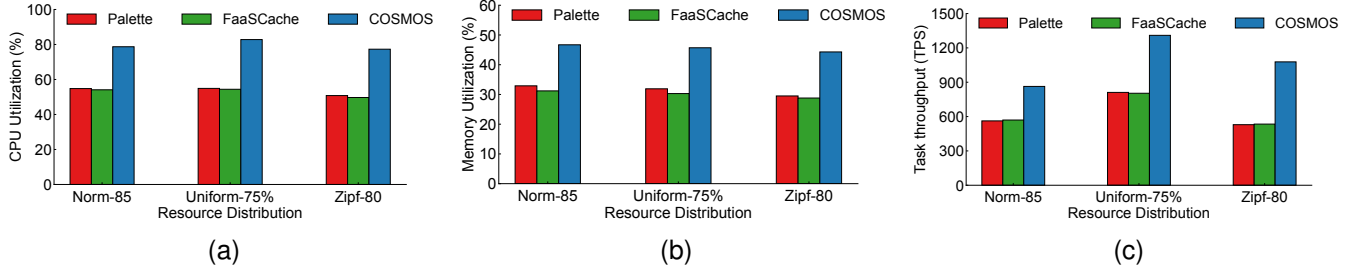


Figure 7. Performance comparison under different cluster resource distributions. (a) CPU Utilization. (b) Memory Utilization. (c) Task throughput.

color, and functions with the same color are co-located on the same server to enhance spatial locality. Inspired by caching systems, FaaSCache employs a Greedy-Dual keep-alive strategy, which assigns a priority to each kept-alive instance and evicts low-priority instances first under resource pressure. To ensure fair comparison and minimize engineering artifacts, we reimplement both baselines in our prototype environment. Other components of the prototype system are kept identical across all evaluated systems.

Metric. We evaluate system performance using CPU/memory utilization and task throughput as primary metrics. To verify that the scheduling algorithm in COSMOS does not lead to task starvation, we also compare the 95th percentile tail latency of task processing. Each data point is collected from a one-hour test run, with the first five minutes of data excluded to eliminate the impact of initial cold starts.

B. Benefits of COSMOS

To better demonstrate the performance of COSMOS under different deployment scenarios, we first compare the overall performance between COSMOS and baselines under different cluster resource distributions, benchmark numbers and workload distributions.

Comparison under different cluster resource distributions. Figure 7 compares the performance of COSMOS and baselines under different cluster resource distributions. We introduce two resource availability patterns: uniform and Zipf distributions. Under the uniform distribution, each server is configured with 75% of its total resources available. In the Zipf-based distribution, the fraction of available resources per server follows a Zipf distribution parameterized with $\alpha = 0.8$ across $n = 8$ resource tiers.

Across all resource distribution patterns, COSMOS consistently improves both CPU and memory utilization, achieving relative improvements of 43.61%–55.53% and 41.41%–67.01% over the baselines, respectively. Furthermore, COSMOS demonstrates performance gains in task throughput, with improvements ranging from 51.52% to 103.71%.

To verify the potential concern of task starvation, we examine the impact on end-to-end performance. As shown in Table II, COSMOS reduces the overall 95th percentile latency by 30.63%–42.42% compared to the baselines. This result indicates that the efficiency gains of COSMOS are not achieved by systematically delaying particular functions. Instead, the improvement in resource utilization and throughput

Table II
TAIL LATENCY COMPARISON BETWEEN COSMOS AND BASELINES.

Resource Distribution	95th Tail Latency (s)		
	Palette	FaaSCache	COSMOS
Norm-85	1071	1104	743
Uniform-75%	1054	1063	729
Zipf-80	1372	1335	790

is achieved alongside enhanced and more predictable end-to-end performance, even when co-locating diverse benchmarks with varying resource demands and execution characteristics.

The performance gains of COSMOS are primarily attributed to two mechanisms. First, the complementary swarming request routing enables individual nodes to serve fewer functions, thereby increasing instance reuse probability. This approach also aligns node workload with resource capacity, avoiding resource waste caused by load-resource mismatch. Second, the monotonic scale-up scheduling policy reduces both resource over-provisioning and cold-start frequency, further boosting overall resource utilization.

Comparison under different benchmark numbers. Next, we evaluate COSMOS and the baselines under different workload scales. Beyond the default configuration of 10 benchmarks, we provide two additional scenarios:

- **5 benchmarks:** A randomly selected subset comprising **video-processing**, **graph-pagerank**, **image-recognition**, **dna-visualization**, and **thumbnailer**. To maintain comparable total load, the invocation rate for each function in this set is doubled;
- **20 benchmarks:** The original 10 benchmarks plus their replicas. Each replica has the same code, resource profile, and runtime as the original but is considered a distinct function, preventing instance reuse between the original-replica pairs. The invocation rate per function is halved to maintain the aggregate user traffic.

As shown in Figure 8, the performance advantage of COSMOS over the baselines grows as the number of benchmarks increases from 5 to 20. Specifically, the improvement in CPU utilization rises from 17.40%–16.90% to 44.89%–44.06%, memory utilization gains increase from 41.81%–43.26% to 55.42%–57.37%, and task throughput improvements expand from 45.22%–51.06% to 75.45%–78.98%.

This scaling trend can be attributed to the complementary swarming routing. With fewer benchmarks, each node already operates on a limited set of functions, leaving limited room for further optimization through instance reuse. In contrast,

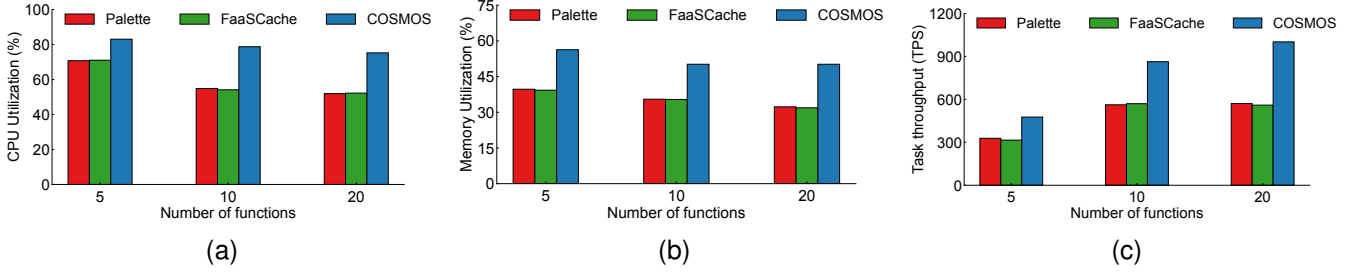


Figure 8. Performance comparison under different function numbers. (a) CPU Utilization. (b) Memory Utilization. (c) Task throughput.

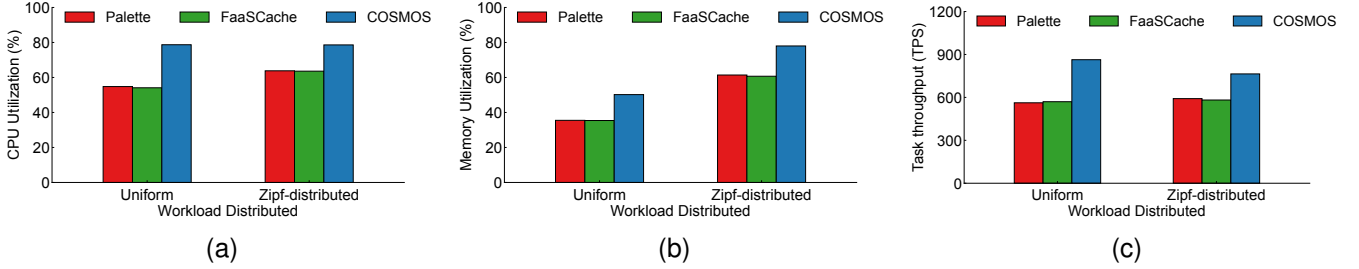


Figure 9. Performance comparison under different workload distributions. (a) CPU Utilization. (b) Memory Utilization. (c) Task throughput.

when the workload is larger and more diverse, the routing reduces the number of distinct functions each node handles. This effect elevates the instance reuse rate, which in turn improves overall efficiency.

Comparison under different workload distributions. Finally, we examine the impact of workload distribution on scheduling performance. In addition to the default resource distribution, we introduce a Zipf-based load pattern. Specifically, we adjust the α parameter of a Zipf distribution (with $n = 10$) to obtain a probability set that approximates the Pareto principle (*i.e.*, the 80/20 rule), where the sum of the two largest probabilities is about 0.8. We then scale the invocation rate of each benchmark according to this distribution, preserving the total aggregate invocation frequency while ensuring that the two most frequently invoked functions—randomly chosen as **thumbnailer** and **video-processing**—collectively account for 80% of all invocations.

It can be observed from Figure 9 that, compared to the default uniform distribution, the optimization margin of COSMOS under the Zipf workload distribution is relatively smaller. Under the Zipf distribution, COSMOS achieves an increase in CPU utilization of 23.20%–23.58%, memory utilization of 27.04%–28.50%, and task throughput of 29.20%–31.34% over the baselines.

This result stems from our experimental configuration: the Zipf distribution adhering to the 80/20 rule implies that only two functions bear the majority of the workload, which naturally grants them more instance reuse opportunities. In contrast, while the real-world workload distribution also exhibits skew, it involves a larger number of high-load functions competing for node resources simultaneously. This competition creates more optimization potential for COSMOS. Moreover, even with only two high-load functions, COSMOS still achieves around 20%–30% higher resource utilization and task throughput than the baselines. This gain is primarily because the monotonic

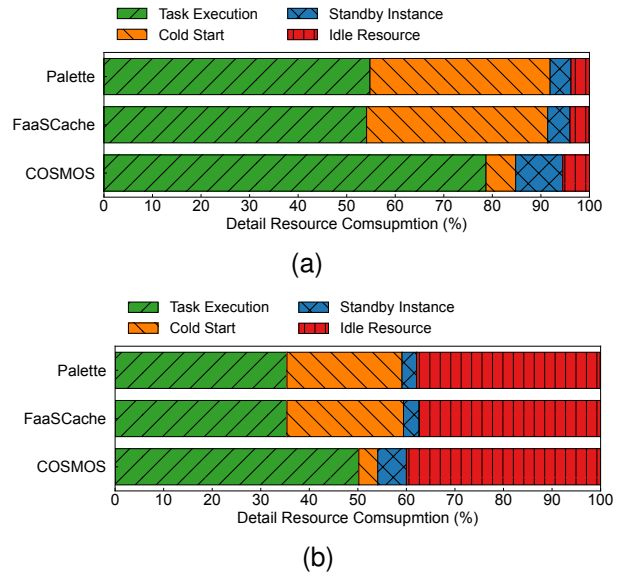


Figure 10. Detail Resource Consumption of COSMOS and baselines. (a) CPU Detail Consumption. (b) Memory Detail Consumption.

scale-up scheduling policy reduces both instance resource over-provisioning and cold-start frequency.

Performance analysis. To further illustrate the reasons behind COSMOS's performance advantage, Figure 10 provides a detailed illustration of the average resource consumption of COSMOS and the baselines during task processing in the default evaluation setting. We categorize CPU and memory resources into the following four types:

- **Task Execution:** resources allocated for processing tasks;
- **Cold Start:** resources consumed during the initialization of instances;
- **Standby Instance:** resources held by ready but currently idle instances;
- **Idle Resource:** available but unallocated resources.

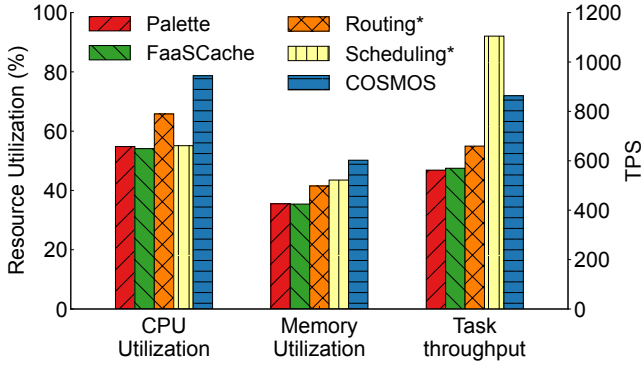


Figure 11. Performance comparison among COSMOS, baselines and simplified variants.

Taking CPU utilization as an example, compared to the baselines, COSMOS increases the resource share for Task Execution by 45.47%–43.61%, for Standby Instance by 110.87%–125.58%, and for Idle Resource by 43.59%–47.37%, while it reduces the share for Cold Start by 83.65%–83.56%. A similar trend is observed for memory utilization.

The resource utilization advantage of COSMOS stems from the reduction in cold starts. This is achieved through two strategies. First, the complementary swarming routing concentrates a narrower set of functions on each node, which increases instance reuse opportunities. Second, the monotonic scale-up scheduling policy reorders requests and dynamically adjusts instance resources, enabling instances to serve varying demands throughout their lifecycle without repeated cold starts or static over-provisioning.

The resources conserved from fewer cold starts not only increase the share for task execution but also expand the pool of idle resources and standby instances, facilitating container reuse and in-place scaling.

C. Effectiveness of COSMOS

In this section, we investigate the impact of the techniques employed by COSMOS. We evaluate COSMOS using the following two simplified variants:

- **Routing***: This variant uses only the complementary swarming request routing strategy, while tasks within each node are scheduled using a simple FIFO policy.
- **Scheduling***: This variant uses only the monotonic scale-up scheduling policy within each node, while user requests are randomly routed across worker nodes.

Figure 11 shows the performance comparison between COSMOS and simplified variants. We also provide the evaluation results of the baselines for reference.

As shown, the performance of the Routing* version lies between the baselines and COSMOS. Compared to the baselines, it achieves improvements of 20.07%–21.63%, 16.90%–17.23%, and 17.31%–15.74% in CPU utilization, memory utilization, and task throughput, respectively. However, it still falls short of COSMOS by 16.39%, 17.33%, and 23.62% in the metrics. These results demonstrate that the complementary swarming routing strategy alone can effectively enhance performance. Furthermore, it requires integration with the monotonic scale-

up scheduling policy for fine-grained, node-internal resource management to achieve better scheduling efficiency.

The performance of the Scheduling* variant reveals a more nuanced trade-off. It underperforms COSMOS in resource utilization, with CPU and memory utilization lower by 13.35% and 29.99%, respectively. Despite this, it achieves a 27.95% higher task throughput. This discrepancy occurs because Scheduling* prioritizes tasks with lower resource demands, shorter execution times, and higher frequencies, without explicitly optimizing for overall server utilization.

When integrated with the complementary swarming routing strategy (as in COSMOS), each node handles a limited set of functions whose resource profiles complement each other. This increases the likelihood that high-resource-demand tasks are selected for instance setup and execution, which in turn raises the execution priority for subsequent requests of the same function (see Section IV-B2). In contrast, without effective load partitioning, a wider variety of instances compete for node resources. In this scenario, resource-intensive instances rely mainly on starvation-avoidance mechanisms and receive only limited execution opportunities. As a result, frequent instance eviction and cold starts occur, which hinders overall resource utilization.

D. Scalability

Finally, we evaluate the scalability of COSMOS. Due to testbed limitations, this section includes both real and simulated experiments. We first demonstrate the performance of COSMOS on a larger-scale cluster. Then we show the system overhead of the COSMOS scheduler on up to 1024 nodes.

Larger-scale cluster experiment. In the larger-scale cluster experiment, we increased the number of worker nodes to 16 and proportionally scaled the invocation frequency, while keeping other conditions constant. Figure 12 shows the performance comparison between COSMOS and the baselines. Compared to the baselines, COSMOS improves CPU utilization by 31.80%–61.75%, memory utilization by 41.95%–53.82%, and task throughput by 40.24%–110.75%. These results align with those observed in the smaller cluster, demonstrating that COSMOS maintains its performance advantage consistently at scale.

System overhead. We next evaluate the overhead of the COSMOS scheduler under varying cluster sizes and user traffic. To do so, we simulate scheduler load for clusters ranging from 64 to 1024 worker nodes. To avoid interference from bursty traffic patterns, we generate invocations at a steady rate instead of using a real-world trace. The experimental results are presented in Figures 13 and 14.

Figure 13 shows the average duration for COSMOS to generate cluster-wide request routing plans. It can be observed that the generation duration increases gradually with cluster scale but remains largely unaffected by invocation frequency. This is because COSMOS scales the overall load proportionally to the cluster size before computing the routing plan. Even with 1024 worker nodes and an invocation rate of 250×10^3 requests per minute, the latency stays below 110 ms. This result indicates that the scheduler can generate routing plans

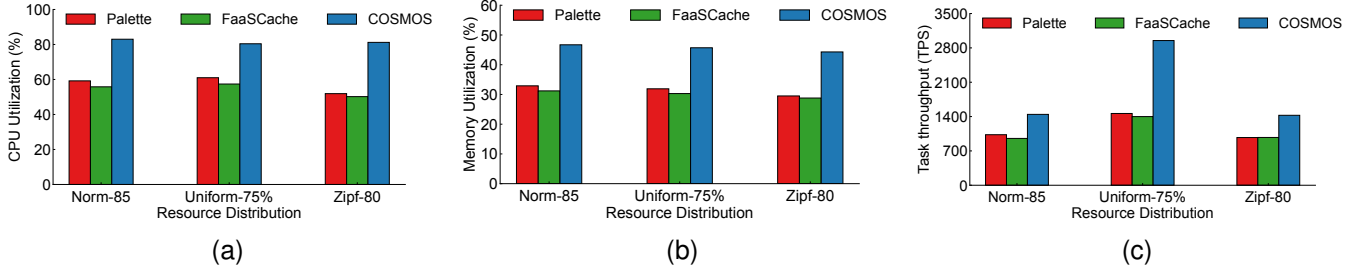


Figure 12. Performance comparison in 16-worker cluster. (a) CPU Utilization. (b) Memory Utilization. (c) Task throughput.

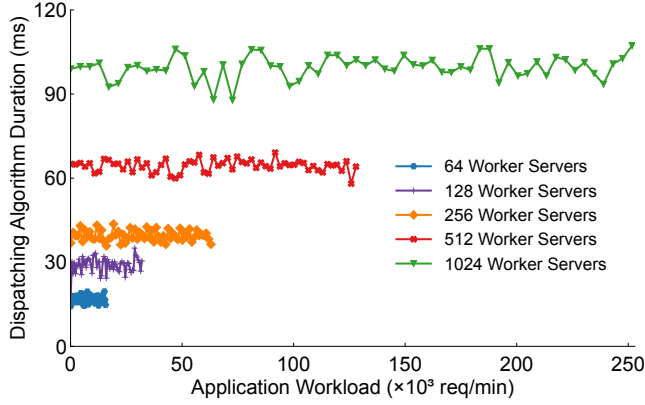


Figure 13. The average duration for COSMOS to generate a routing plan.

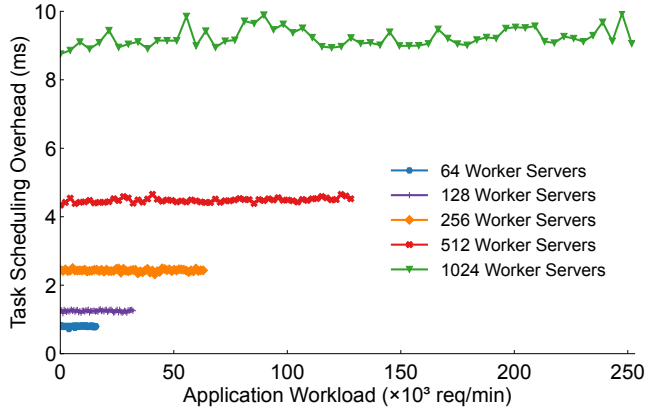


Figure 14. The average scheduling overhead of COSMOS.

promptly, even in large-scale clusters, demonstrating the practical scalability of COSMOS.

Figure 14 shows the scheduling overhead for individual tasks under different scenarios. It should be noted that routing plan generation is performed by a separate background process within the scheduler. The main scheduling process only receives the resulting routing plan and assigns tasks accordingly. Therefore, only the scheduling overhead contributes to the additional latency in the end-to-end task execution path. As shown, even under a heavy load of 1024 worker nodes, the extra scheduling delay added by COSMOS remains below 10 ms. This overhead is negligible for the vast majority of FaaS scenarios, confirming that the scheduling logic of COSMOS is lightweight enough for production-scale deployment.

VII. RELATED WORK

Serverless runtime resource configuration. Optimizing per-instance resource allocation for serverless functions is a well-explored area of research. A prevalent method is profile-

guided static tuning [15], [30], involving benchmarking each function across various resource setups (*e.g.*, memory sizes, CPU cores) to establish a latency-vs-cost curve. The least costly configuration that fulfills the service level objectives (SLOs) is chosen for deployment. Other works [5], [20]–[24] go further to optimize resource allocation for serverless workflows, by modeling the relationship between E2E latency and individual function resources. Cloud providers’ tools (*e.g.*, AWS’ power tuning tool [28]) employ this technique to assist users in optimizing memory allocations. Unfortunately, all of them assume a static and uniform resource allocation per function, ignoring the significant per-request resource usage variations. Such static allocation can lead to over-provisioning for lightweight requests, resulting in suboptimal utilization. By enabling dynamic resizing of serverless function resources, our work addresses this limitation, allowing more efficient resource usage.

Instance Orchestration and Placement. This line of work focuses on managing the lifecycle, quantity, and location of function instances, typically under a static per-instance resource model. A primary concern is optimizing the instance count to balance cost and cold-start latency. Strategies include keeping instances alive in a keep-alive window [15], predictive pre-warming based on demand patterns [14], [23], and dynamic scaling based on real-time metrics [24], [31]–[33]. To improve placement efficiency, recent work considers several dimensions: i) Cache awareness, which reuses instance states (*e.g.*, through copy-on-write cloning) to reduce startup overhead [34], [49]; ii) Cost-optimized placement, which matches functions to specialized or lower-cost hardware (*e.g.*, spot instances, GPUs) to reduce deployment expenses [14], [50]; and iii) Data affinity, which co-locates dependent tasks in serverless workflows to minimize data transfer, as seen in systems like Wukong [35], FaaSFlow [51], ORION [23], and Ditto [20]. However, a fundamental premise shared by these approaches is the assignment of a static, uniform resource capacity to each function instance. In contrast, COSMOS explicitly accounts for resource heterogeneity, aligning complementary workload resource profiles with node capacity through instance placement, thereby minimizing fragmentation and resizing overhead.

Request and Task Scheduling. For request and task scheduling, prior work has focused on improving scheduling efficiency, load balancing, and resource cost optimization. For instance, Hermod is cost-, load-, and locality-aware to optimize consolidation and reduce cold starts [9]. XFaaS defers delay-tolerant functions and dispatches calls globally to handle load spikes [52]. Atoll

employs a tiered architecture with semi-global schedulers and sandbox-aware routing [53]. Kaffes et al. advocate for a centralized, core-granular scheduler to eliminate queue imbalance [54]. Golgi minimizes costs through carefully managed overcommitment based on historical usage [55]. However, these scheduling policies largely ignore the per-request resource demand variations. Our monotonic scale-up scheduling policy reorders function requests to ensure resource demands increase monotonically, thereby amortizing cold start overhead and avoiding over-provisioning.

VIII. CONCLUSION

This paper presents COSMOS, a serverless computing system designed to address the resource utilization challenges caused by heterogeneous function resource demands. First, COSMOS co-locates functions at the cluster level with complementary swarming request routing, which not only improves overall server utilization but also increases instance reuse opportunities by narrowing the function serving set per node. Second, with monotonic scale-up scheduling, COSMOS aligns request processing ordering with the in-place scaling capability of Kubernetes pods. This allows instance resource allocation to be dynamically adjusted according to actual request demands, effectively reducing cold starts without over-provisioning. As demonstrated in the evaluation, COSMOS shows significant improvements over the baselines in both resource utilization and task throughput, while adding negligible system overhead and maintaining scalable performance.

ACKNOWLEDGMENTS

This research is supported by the Huawei Peking OS Innovation Lab.

REFERENCES

- [1] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski, "The rise of serverless computing," *Communications of the ACM*, 2019.
- [2] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Menezes Carreira, K. Krauth, N. Yadwadkar, J. Gonzalez, R. A. Popa, I. Stoica, and D. A. Patterson, "Cloud programming simplified: A Berkeley view on serverless computing," Tech. Rep., 2019.
- [3] J. Schleier-Smith, V. Sreekanti, A. Khandelwal, J. Carreira, N. J. Yadwadkar, R. A. Popa, J. E. Gonzalez, I. Stoica, and D. A. Patterson, "What serverless computing is and should become: the next phase of cloud computing," *Communications of the ACM*, 2021.
- [4] Microsoft, "Serverless web application - azure architecture center," <https://learn.microsoft.com/en-us/azure/architecture/web-apps/serverless/architectures/web-app>, 2025, accessed: 2025-10-10.
- [5] Q. Pu, S. Venkataraman, and I. Stoica, "Shuffling, fast and slow: Scalable analytics on serverless infrastructure," in *16th USENIX symposium on networked systems design and implementation (NSDI 19)*, 2019, pp. 193–206.
- [6] A. Klimovic, Y. Wang, P. Stuedi, A. Trivedi, J. Pfefferle, and C. Kozyrakis, "Pocket: Elastic ephemeral storage for serverless analytics," in *USENIX OSDI*, 2018.
- [7] A. Ali, R. Pincioli, F. Yan, and E. Smirni, "Batch: Machine learning inference serving on serverless platforms with adaptive batching," in *The International Conference for High Performance Computing, Networking, Storage, and Analysis*, 2020.
- [8] J. Thorpe, Y. Qiao, J. Eyolfson, S. Teng, G. Hu, Z. Jia, J. Wei, K. Vora, R. Netravali, M. Kim, and G. H. Xu, "Dorylus: Affordable, scalable, and accurate gnn training with distributed cpu servers and serverless threads," in *USENIX OSDI*, 2021.
- [9] K. Kaffes, N. J. Yadwadkar, and C. Kozyrakis, "Hermod: principled and practical scheduling for serverless functions," in *ACM Symposium on Cloud Computing*, 2022.
- [10] Knative, "Knative," <https://knative.dev/>, 2025, accessed: 2025-10-10.
- [11] O. Ltd., "Openfaas," <http://openfaas.com/>, 2025, accessed: 2025-10-10.
- [12] A. Fuerst and P. Sharma, "Faas-cache: keeping serverless computing alive with greedy-dual caching," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21, New York, NY, USA, 2021, pp. 386–400.
- [13] H. Yu, R. Basu Roy, C. Fontenot, D. Tiwari, J. Li, H. Zhang, H. Wang, and S.-J. Park, "Rainbowcake: Mitigating cold-starts in serverless with layer-wise container caching and sharing," in *ACM ASPLOS*, 2024.
- [14] R. B. Roy, T. Patel, and D. Tiwari, "Icebreaker: warming serverless functions better with heterogeneity," in *Proceedings of the 2022 ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Lausanne, Switzerland, 2022, pp. 753–767.
- [15] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proceedings of the 2020 USENIX Annual Technical Conference*. California, USA, 2020, pp. 205–218.
- [16] T. A. S. Foundation, "Apache openwhisk," <https://openwhisk.apache.org/>, 2025, accessed: 2025-10-10.
- [17] A. W. Services, "Configure lambda function memory," <https://docs.aws.amazon.com/lambda/latest/dg/configuration-memory.html>, 2025, accessed: 2025-10-10.
- [18] Microsoft, "Azure functions hosting options," <https://learn.microsoft.com/en-us/azure/functions/functions-scale>, 2025, accessed: 2025-10-10.
- [19] Google, "Configure memory limits for services," <https://cloud.google.com/run/docs/configuring/services/memory-limits>, 2025, accessed: 2025-10-10.
- [20] C. Jin, Z. Zhang, X. Xiang, S. Zou, G. Huang, X. Liu, and X. Jin, "Ditto: Efficient serverless analytics with elastic parallelism," in *Proceedings of the ACM SIGCOMM 2023 Conference*. New York, NY, USA, 2023, pp. 406–419.
- [21] Z. Zhang, C. Jin, and X. Jin, "Jolteon: Unleashing the promise of serverless for serverless workflows," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 167–183.
- [22] H. Zhang, Y. Tang, A. Khandelwal, J. Chen, and I. Stoica, "Caerus: NIMBLE task scheduling for serverless analytics," in *Proceedings of the 2021 USENIX Symposium on Networked Systems Design and Implementation*. Boston, MA, USA, 2021, pp. 653–669.
- [23] A. Mahgoub, E. B. Yi, K. Shankar, S. Elnikety, S. Chatterji, and S. Bagchi, "ORION and the three rights: Sizing, bundling, and prewarming for serverless dags," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation*. Carlsbad, CA, USA, 2022, pp. 303–320.
- [24] Z. Zhou, Y. Zhang, and C. Delimitrou, "AQUATOPE: qos-and-uncertainty-aware resource management for multi-stage serverless workflows," in *Proceedings of the 2023 ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. BC, Canada, 2023, pp. 1–14.
- [25] O. Alipourfard, H. H. Liu, J. Chen, S. Venkataraman, M. Yu, and M. Zhang, "CherryPick: Adaptively unearthing the best cloud configurations for big data analytics," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pp. 469–482.
- [26] A. Klimovic, H. Litz, and C. Kozyrakis, "Selecta: Heterogeneous cloud storage configuration for data analytics," in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 759–773.
- [27] A. Mahgoub, A. M. Medoff, R. Kumar, S. Mitra, A. Klimovic, S. Chatterji, and S. Bagchi, "OPTIMUSCLOUD: Heterogeneous configuration optimization for distributed databases in the cloud," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pp. 189–203.
- [28] A. Casalboni, "Aws lambda power tuning," 2024. [Online]. Available: <https://github.com/alexcasalboni/aws-lambda-power-tuning>
- [29] Kubernetes, "Configure pods and containers — kubernetes." [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/> Accessed: 2025-10-07.
- [30] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *2018 USENIX annual technical conference (USENIX ATC 18)*, 2018, pp. 133–146.
- [31] AWS Lambda, "Monitoring concurrency," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html#managing-provisioned-concurrency>
- [32] AWS, "Lambda function scaling," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/scaling.html>

- [33] Knative, “Configuring concurrency,” 2024. [Online]. Available: <https://knative.dev/docs/serving/autoscaling/concurrency/>
- [34] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: rapid task provisioning with serverless-optimized containers,” in *Proceedings of the 2018 USENIX Annual Technical Conference*. Boston, MA, USA, 2018, pp. 57–70.
- [35] B. Carver, J. Zhang, A. Wang, A. Anwar, P. Wu, and Y. Cheng, “Wukong: a scalable and locality-enhanced framework for serverless parallel computing,” in *Proceedings of the 2020 ACM Symposium on Cloud Computing*. Virtual Event, 2020, pp. 1–15.
- [36] A. Joosen, A. Hassan, M. Asenov, R. Singh, L. Darlow, J. Wang, Q. Deng, and A. Barker, “Serverless cold starts and where to find them,” in *Proceedings of the Twentieth European Conference on Computer Systems*, ser. EuroSys ’25, p. 938–953.
- [37] Kubernetes, “Container runtime interface (cri) — kubernetes.” [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/cri/> Accessed: 2025-10-07.
- [38] Kubernetes, “Open-source system for automating deployment, scaling, and management of containerized applications,” 2024. [Online]. Available: <https://kubernetes.io>
- [39] MinIO, “Exascale AI Data Store,” 2025. [Online]. Available: <https://www.min.io>
- [40] SciPy, “Fundamental algorithms for scientific computing in Python,” 2024. [Online]. Available: <https://scipy.org>
- [41] Knative, “Configuring scale bounds,” 2024. [Online]. Available: <https://knative.dev/docs/serving/autoscaling/scale-bounds/>
- [42] M. Abdi, S. Ginzburg, X. C. Lin, J. Faleiro, G. I. Chaudhry, I. Goiri, R. Bianchini, D. S. Berger, and R. Fonseca, “Palette load balancing: Locality hints for serverless functions,” in *Proceedings of the Eighteenth European Conference on Computer Systems*. Rome, Italy, 2023, pp. 365–380.
- [43] M. Copik, G. Kwasniewski, M. Besta, M. Podstawski, and T. Hoefler, “Sebs: A serverless benchmark suite for function-as-a-service computing,” in *Proceedings of the 22nd International Middleware Conference*, ser. Middleware ’21, p. 64–78.
- [44] S. P. C. L. (SPCL), “Sebs: Serverless benchmark suite,” 2024. [Online]. Available: <https://github.com/spcl/serverless-benchmarks/tree/master>
- [45] R. G. Gallager, *Stochastic Processes: Theory for Applications*. Cambridge University Press, 2013.
- [46] S. S. SHAPIRO and M. B. WILK, “An analysis of variance test for normality (complete samples),” *Biometrika*, vol. 52, no. 3-4, pp. 591–611, dec 1965.
- [47] H. W. Lilliefors, “On the kolmogorov-smirnov test for normality with mean and variance unknown,” *Journal of the American Statistical Association*, vol. 62, no. 318, pp. 399–402, 1967.
- [48] “CloudLab,” <https://www.cloudlab.us>.
- [49] Z. Li, L. Guo, Q. Chen, J. Cheng, C. Xu, D. Zeng, Z. Song, T. Ma, Y. Yang, C. Li *et al.*, “Help rather than recycle: Alleviating cold startup in serverless computing through {Inter-Function} container sharing,” in *2022 USENIX annual technical conference (USENIX ATC 22)*, 2022, pp. 69–84.
- [50] D. Du, Q. Liu, X. Jiang, Y. Xia, B. Zang, and H. Chen, “Serverless computing on heterogeneous computers,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, oriLausanne, Switzerland, pp. 797–813.
- [51] Z. Li, Y. Liu, L. Guo, Q. Chen, J. Cheng, W. Zheng, and M. Guo, “Faas-flow: enable efficient workflow execution for function-as-a-service,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Lausanne, Switzerland, 2022, pp. 782–796.
- [52] A. Sahraei, S. Demetriou, A. Sobhgol, H. Zhang, A. Nagaraja, N. Pathak, G. Joshi, C. Souza, B. Huang, W. Cook *et al.*, “Xfaas: Hyperscale and low cost serverless functions at meta,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 231–246.
- [53] A. Singhvi, A. Balasubramanian, K. Houck, M. D. Shaikh, S. Venkataraman, and A. Akella, “Atoll: A scalable low-latency serverless platform,” in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC ’21, p. 138–152.
- [54] K. Kaffes, N. J. Yadwadkar, and C. Kozyrakis, “Centralized core-granular scheduling for serverless functions,” in *Proceedings of the ACM Symposium on Cloud Computing*, ser. SoCC ’19, p. 158–164.
- [55] S. Li, W. Wang, J. Yang, G. Chen, and D. Lu, “Golgi: Performance-aware, resource-efficient function scheduling for serverless computing,” in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, ser. SoCC ’23, p. 32–47.



Yunshan Jia received the BS degree from Peking University in 2021. She is currently working toward the PhD degree with the School of Computer Science, Peking University. She is currently working on serverless workflow orchestration. Her research interests include cloud computing and wireless communication.



Junyi Shu (Member, IEEE) received the PhD degree from Peking University in 2025. He is currently a postdoctoral researcher with the Computer Science Department, University of California, Los Angeles. His research interests include cloud computing and networked systems.



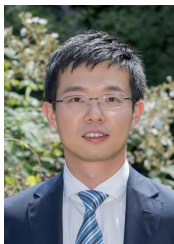
Chiheng Lou received the B.E. degree in computer science from the University of Chinese Academy of Sciences in 2023. He is currently pursuing the Ph.D. degree with the School of Computer Science, Peking University. His research interests include machine learning systems and serverless computing.



Fangyue Liu received the BS degree from Peking University in 2024. He is currently working toward the PhD degree with the School of Computer Science, Peking University. He is currently working on inference systems for large models. His research interests include cloud computing and machine learning systems.



Chenbo Bi is currently pursuing his Bachelor's degree in Computer Science and Technology at Yuanpei College, Peking University. He is currently working on performance optimization for large language models. His research interests include cloud computing and machine learning systems.



Xin Jin (Senior Member, IEEE) received the PhD degree from Princeton University in 2016. He is currently an associate professor (with Tenure) with the School of Computer Science, Peking University. His research interests include computer systems, networking, and cloud computing.